Municipal Research

Repository: Mihagley/Scraping_Swedish_elderly_care. For running and storing real research directly in GitHub, see research/README.pdf. The **Research pilot** workflow collects or classifies the six-municipality seed set and commits the raw files, cleaned text, Excel output and audit back to the repository.

A configurable Python pipeline for discovering municipal sources, preserving HTML/PDF evidence, extracting text, independently coding it with structured LLM responses, verifying quotations, and exporting an auditable Excel workbook.

The default question concerns formal Swedish-language requirements for elderly-care staff before 1 January 2025. The browser interface includes all 290 municipalities with websites and codes matched against SKR, plus all 21 regions. The national list is an input registry, not a completed study. The original six-city list is preserved in `examples/municipalities_pilot.csv`.

Quick start

Requires Python 3.11 or later. From this directory:

```
python -m venv .venv
# Windows PowerShell: .\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.lock.txt
python -m pip install --no-deps -e .
municipal-research demo --run-dir runs/demo
municipal-research verify-run runs/demo
```

The demo uses **invented municipalities and policies**. HTML, a two-page PDF, the real extraction libraries, the real OpenAI SDK against a mocked HTTP transport, structured parsing, adjudication, quote checks, caching and Excel export all run. It makes no external requests, needs no key, and is not substantive research. The delivered project also includes an already-generated example in `example-output/`, with its Excel workbook, raw fixtures, cleaned text and complete model audit. For a lighter production installation, use `python -m pip install .` instead. The demo and tests need development dependencies: `python -m pip install -e '.[dev]'`.

For live research, set `OPENAI_API_KEY` in your shell or secret manager. The program does not automatically read `.env`; `.env.example` documents the variable.

```
$env:OPENAI_API_KEY = "your-key"
municipal-research run --config examples/language_requirements.yaml --municipalities examples/
municipalities.csv --run-dir runs/pilot --limit 1
```

On macOS/Linux use `export OPENAI_API_KEY="your-key"` before the same command. Check the YAML's model names against the models available to your API project. `gpt-5-mini` is an example default, not a claim about the newest or best model. Pinned model snapshots, where available, are preferable for a study. The live pipeline requires outbound access to `api.openai.com` and the municipality domains. Configure network permissions in the environment in which you run it.

Start with one municipality and inspect its **Summary**, **Evidence**, and **Errors** sheets before scaling. Exit code **0** means completed without flagged review items; **2** means exported results

contain gaps or review items; **1** means a fatal error. A successful run is never a claim of exhaustive historical coverage.

Configure the research

`examples/language_requirements.yaml` contains the question, exclusive historical cutoff, detailed instructions, classification labels, extracted attribute definitions, discovery queries, keywords, network limits and model settings. `examples/generic.yaml` demonstrates a different research question with different labels and attributes.

The label names become a JSON Schema enum. Quote YAML keys such as `'Yes'` and `'No'` because YAML 1.1 otherwise treats them as booleans. Unknown configuration fields fail validation instead of being silently ignored.

Each label defines:

- `definition`: the coding rule supplied to every model pass.
- `requires_evidence`: require a verified finding quotation, default true.
- `requires_before_cutoff`: require pre-cutoff temporal classification and a verified timing quotation, default false.
- `conclusive`: whether a verified label can become a municipality summary, default true.

Set `unknown_label` to a non-conclusive category. `Review` is reserved for conflicts at the municipality level. Research attributes are configurable name/description pairs; returned attributes require valid quote indices. Dates and scopes require timing and scope quotations respectively.

The municipality CSV has `id,name,domains,seeds`. IDs remain strings, including leading zeroes. Separate domains and seed URLs with semicolons. Use hostnames without schemes or wildcards in `domains`; subdomains are automatically included. Add a document-hosting domain explicitly if an official source uses an external document archive. Sources outside this allowlist are not downloaded. The supplied national registry covers all 290 municipalities; see `examples/registry_sources/README.pdf` for sources and reproducible validation.

Discovery and collection

`discovery.provider` supports:

- `hybrid`: OpenAI web-search candidates plus sitemaps and bounded link crawling.
- `openai`: search candidates and explicitly supplied seeds/homepages.
- `crawl`: seeds, homepages, robots.txt sitemap hints, sitemap indexes and links.

Queries may use `{municipality}`, `{domain}`, and `{question}`. Search results are URL candidates only: classification uses the independently downloaded source, never the search answer or snippet. URLs are taken from tool sources/citations, not freeform model prose. Sitemaps and links are ranked with configurable keywords; ranking is a discovery heuristic, not a factual classifier. Explicit seeds receive highest priority.

Collection respects robots.txt by default, checks allowed hosts at every redirect, blocks non-public resolved addresses, observes per-host spacing and robots crawl-delay, limits

response size, and retries network/transient HTTP failures. HTTP retries honor Retry-After. A robots 401/403 denies access; 404/410 means no rules; retrieval failures fail closed. Ordinary 4xx errors are not retried. No login, access bypass, browser automation, or JavaScript rendering is included. DNS checks are defense in depth, not a replacement for an outbound firewall against adversarial DNS rebinding.

Raw files preserve the received, HTTP-decoded entity body: PDF bytes or HTML bytes, before text cleanup. Redirect destination, retrieval time, media type and SHA-256 are recorded. The HTTP cache also retains downloaded sitemap and robots bodies.

HTML extraction uses Trafilatura with tables and a documented BeautifulSoup fallback. PDF extraction uses pypdf per physical page. Sparse or unreadable pages produce review warnings; **OCR is not performed**. Image-only PDFs require an explicit OCR extension or manual review. PDFs with columns or tables can have imperfect reading order; check the preserved PDF before relying on a conclusion. Very large or hostile documents should be processed in an externally resource-limited environment.

To collect without classification, set the provider to `crawl` for a no-API collection run, then use `--collect-only`. With `hybrid`/`openai`, discovery still uses the paid API. Collected results have category `NotClassified`; they are not research conclusions.

Classification and verification

Each saved text is divided into overlapping character windows. Every window is analyzed unless `max_chunks_per_document` is reached, in which case coverage is flagged. There is no silent first-N truncation. The default is three fresh, stateless model calls; each sees the source and codebook, but no other pass's output. `llm.models` can rotate different OpenAI models across the passes. This provides procedural independence, not a guarantee of independent statistical errors.

The current OpenAI Responses API's Pydantic structured-output pattern is used via `client.responses.with_raw_response.parse(..., text_format=Schema)`. Its public raw response wrapper allows saving API JSON before local schema validation, including refusals, incomplete responses and malformed structured outputs. The schema forbids extra fields. Classification has no tools, conversation history or external knowledge.

1. Mechanically verify every candidate's quotations and required supporting fields.
2. Declare consensus only if all passes are valid and agree on category, chronology, effective date, scope and attribute values; agreeing labels alone are insufficient.
3. Have a separate verifier check whether source passages support the selected claims.
4. On disagreement, invalid quotations or verifier rejection, ask an adjudicator to reassess the source and candidate audit. Verify its output mechanically and with another semantic-verifier call.
5. If verification still fails, retain all attempts in the audit and abstain with the configured unknown category and `needs_review=true`.

The verifier and adjudicator models are separately configurable. They can still share biases and make mistakes. Their acceptance is an automated review result, not proof. The pipeline asks for concise evidence-based justifications, not hidden reasoning.

The default codebook distinguishes Yes, No, Pilot, Later, Proposed, Unclear, NoEvidence and Irrelevant. A test-method pilot is distinct from a pilot of the underlying requirement. Retrieval/publication dates do not establish implementation dates. SFI, Swedish courses and CEFR levels are not assumed equivalent.

At municipality level, one conclusive category is retained; competing conclusive categories, dates, scopes or attribute values produce `Review`. Unknown values do not override positive evidence and are retained in Classifications. This deliberately conservative merge does not resolve cross-document contradictions automatically. Evidence needing several separate chunks/documents to establish a claim may remain uncertain: there is no unconstrained cross-document synthesis step. Increase overlap or review the full sources where needed.

Exact evidence locations

Quotations match either exactly or after whitespace-only normalization. Case changes, paraphrases, translations, ellipses and fuzzy similarity are not accepted. A quotation must occur inside the chunk actually supplied to the model. All matching occurrences are recorded; the pipeline never silently chooses the first repeated phrase.

Location fields use:

- **Characters:** zero-based Unicode code-point offsets, end exclusive, in `text.txt`.
- **Lines:** one-based lines in `text.txt`, shown alongside numbers in `text.lines.txt`.
- **PDF pages:** one-based physical pages; page-local characters use the same convention.
- **HTML pages:** null, since a webpage has no physical PDF page number.

Page boundaries in PDF text use `\n\f\n`. Offsets are not raw HTML byte positions, DOM selectors, PDF bounding boxes or printed page labels. Every location includes the actual matched substring, and the text hash links it to the saved text version.

Outputs and replay

Each new run gets its own directory; existing nonempty directories are refused.

```
runs/pilot/
  results.xlsx              Ten formatted, filtered sheets with frozen headers
  results.json              Complete structured dataset (authoritative)
  manifest.json             Version, environment, config/input/code and artifact hashes
  config.resolved.json       Full resolved configuration
  municipalities.json      Exact input records
  audit.jsonl               Timestamped events, retries, API usage and errors
  pipeline.log             Human-readable logging
  sources/<id>/            Raw source, download metadata, clean text, numbered text, page map
  chunks/<hash>.json        Exact windows supplied to classification
  units/<hash>.json         Final decision and complete per-window pass audit
  llm/<hash>/               Prompts, schemas, API responses and API error metadata
```

Excel sheets: Summary, Classifications, Evidence, Attributes, Sources, Discovery, Pass_Audit, Errors, API_Usage, Methodology. Evidence includes final and intermediate quotes; filter `stage=final` for accepted outputs. Attribute quote indices refer to the final evidence list for the same chunk. The summary's quote count counts accepted quote entries, not unique phrases or independent sources. Text that resembles a formula is exported literally. Excel display cells longer than 32,000 characters are marked as truncated; complete data remains in JSON. Source

URLs and relative source paths are included. A quote may appear repeatedly because windows overlap or different purposes use the same supporting passage.

The shared `data/cache/` contains content-addressed HTTP bodies, URL metadata and LLM responses. Cache entries do not expire automatically. Reruns reuse them, preserving the original retrieval time. Use a new run directory with `--refresh` to retrieve fresh sources and make fresh model calls. Previously saved run directories are unchanged.

```
# Rerun using the exact same config and input, with no HTTP or API access:
municipal-research run --config examples/language_requirements.yaml --municipalities examples/
municipalities.csv --run-dir runs/replay --offline --limit 1

# Verify the saved run, or export a new workbook without rerunning models:
municipal-research verify-run runs/pilot
municipal-research export runs/pilot --output exports/pilot.xlsx
```

Keep both the run and its cache for full pipeline replay. Alternatively use `export` with just the run directory. A cache miss in offline mode becomes an explicit gap; it never falls back to the network. Interrupted runs retain finished units. To resume, use the same cache and inputs with a new run directory: cached HTTP/LLM work is reused. The original partial run is preserved. One process should own a run/cache at a time.

`verify-run` detects missing or modified artifacts against the manifest. It is an integrity check, not a digital signature or validation of the substantive conclusions. Fresh network searches and fresh model calls are not bit-for-bit deterministic. Saved inputs, responses and dependency versions make a particular run auditable and replayable.

Limits, cost and extending the project

`max_documents` limits attempted document downloads per municipality; robots and sitemap requests have separate limits. `max_sitemap_urls`, `max_depth` and `max_chunks_per_document` also bound work and are reported when reached. Failed downloads count against the document budget. All limits are configurable.

With P independent passes, one chunk uses P+1 model calls for verified consensus, P+2 for direct adjudication, or P+3 if consensus is rejected and then adjudicated. Discovery calls are additional. `llm.max_calls` is a per-run invocation limit; SDK internal retries may add requests, and web-search calls can have additional cost. Use API-project spend limits for a monetary cap. `API_Usage` separates cached responses from new invocations; do not count cached token metadata as new usage.

Production usage needs an API key and billable API access. No live API request or substantive municipality study is included in the demonstration. To extend the pipeline, implement a search provider in `discovery.py`, an extraction route in `extraction.py`, or a provider adapter with the Gateway `parse/search` interface. Keep deterministic quote checks and the audit contract at those boundaries.

Development and GitHub

```
python -m pip install -e '.[dev]'
python -m ruff check .
python -m ruff format --check .
python -m pytest --cov=municipal_research --cov-report=term-missing
python -m build
```

Tests use local fixtures and mocked HTTP, not live sites or paid API calls. They cover SDK request/schema construction, refusals, incomplete/malformed outputs, strict quote locations, Swedish Unicode, PDFs, chunk coverage, robots, retries, allowlists, caching, independent passes, adjudication, summary conflicts, Excel injection and end-to-end replay.

`requirements.lock.txt` records the exact resolved direct and transitive versions used for validation; `pyproject.toml` supplies compatible dependency bounds. The lock is a version lock, not a wheel-hash lock. CI is configured for Python 3.11–3.13 on Linux and Windows. Local validation does not imply that the remote CI matrix has already run.

The project is ready to add to a Git repository. `.gitignore` excludes keys, caches and research runs. Review source redistribution rights and any personal information before publishing collected materials. No repository is created or published automatically.

Documentation references

Implementation patterns were checked against the official documentation on 2026-09-16:

- OpenAI structured outputs and Pydantic parsing
- OpenAI web search, domain filtering and sources
- OpenAI SDK libraries

See `docs/architecture.pdf` for the module map and the boundaries of mechanical and semantic verification.

Browser interface: kommun and region

A Streamlit interface supports keyword/query entry, research questions, individual kommun/region selection, CSV and YAML uploads, offline demos, live research, job activity, and Excel/JSON downloads. The interface is in Swedish and includes all 290 municipalities and all 21 regions. See interface and hosting instructions.

```
python -m pip install -r requirements.txt
python -m streamlit run streamlit_app.py
```

Local AI with Ollama

Classification, semantic verification and adjudication can use a downloaded local model through Ollama's native structured-output API. Set `llm.provider: ollama`; `examples/local_llama.yaml` is a complete Swedish example using `llama3.1:8b`. All classification roles use the selected local model in the interface. Quote checks, raw-response audits, cache integrity checks and offline replay remain active.

Install/start Ollama, then run `ollama pull llama3.1:8b` (or choose another locally installed model suitable for your machine). In the interface choose **Lokal AI via Ollama**. The default endpoint is `http://localhost:11434`; use an origin without `/api` or `/v1`. Only loopback endpoints are accepted. The connection-test button checks availability and installed model names; it does not download a model.

Ollama does not replace the scraper's OpenAI web-search tool. With discovery set to `crawl`, classification can run without an OpenAI API key. Explicitly selecting `hybrid`/`openai` still uses paid

OpenAI web search and requires that key. Sources are still fetched from the public sites selected by the user. No automatic cloud fallback is used for failed local classifications. The offline demo always uses its fixed mocked model responses, even with a local-provider configuration.

See `docs/lokal-ai.pdf` for Swedish setup and limitations. Local outputs are validated against the same Pydantic schema; malformed, incomplete and token-limited responses remain review items. Cache keys include the local provider, origin, model, context, full prompt/schema, role and generation settings. Use a fresh cache or `--refresh` when replacing the weights behind an existing Ollama model tag.

The browser interface automatically uses crawl discovery whenever no OpenAI key is entered. Entering a key unlocks OpenAI discovery options; it does not start a paid search automatically. Classification remains an explicit choice: use Ollama, supply an OpenAI key, or select collection only. Swedish question labels use Utredningsfråga.